

2026

Samarbejde om industrielt IoT-projekt.



Micro Technic

Samarbejde om industrielt IoT-projekt.

26-08-2026

Indhold

Samarbejde mellem UCL og Micro Technic: IoT-projekt 3. semester	3
Scope vs out of scope	3
Ansvarsfordeling	3
Opsætning af cloud	4
Kravspecifikationer for Zephyr	4
RTOS og toolchain	4
Udviklingsmiljø	4
Projekt C: Udviklingsmiljø (Nordic Thingy:91 X)	5
Zephyr konfiguration med Kconfig	5
Arkitektur	7
Mappestruktur	7
Protokoller	8
Logging	8
Fælles for dataformat	8
Projekt A – Espressif ESP32C6-devkitc	9
ESP32C6 m. ekstern cellulært modul – cellulær kommunikation	9
Kravspecifikationer	9
Dataopsamling	9
Opsamling gennem vilkårlig metode	9
Projekt B	10
Industrial IoT ESP32 PLC – Ethernet kommunikation	10
Kravspecifikationer	10
Dataopsamling	10
Projekt C	11
Nordic Thingy:91 X – Cellulær kommunikation	11
Kravspecifikationer	11
Dataopsamling	11
Opsamling gennem vilkårlig metode	11

Nice to have 11

Getting started 12

Overholdelse af gældende standarder 12

62443..... 12

Cyber Resilience Act 12

Samarbejde mellem UCL og Micro Technic: IoT-projekt 3. semester

Dokumentet har til formål at beskrive kravspecifikationer for de projekter, som er udbudt til studerende på UCL. Først vil generelle kravspecifikationer for Zephyr RTOS, udviklingsmiljø, arkitektur og dataopsamling blive beskrevet, og derefter vil kravspecifikationer relateret til de specifikke projekter blive præsenteret.

Alle projekter inkluderer Zephyr RTOS, og er derfor generelt specificeret for alle projekter.

Scope vs out of scope

Projektet omfatter:

- Udvikling af embedded firmware baseret på Zephyr RTOS.
- Kommunikation med cloud-platform via MQTT.
- Dataopsamling
 - Målinger gennem Modbus eller udleverede sensorer.
- Simpel cloud instans til modtagelse af data.

Projektet omfatter ikke:

- Avanceret certifikat arkitektur som PKI (Public Key Infrastructure).
- Avanceret cloud drift, skalerbarhed og compliance for cloud.

Ansvarsfordeling

Micro Technic stiller til rådighed:

- Grundig facilitering af udbudte projekter.
- Basis referencer/sample projekt som eksempler på projektstruktur.
- Livslinje ved teknisk svære problemstillinger.

Studerende er ansvarlige for:

- Implementering af Zephyr applikation på MCU (Micro Controller Unit).
- Netværskommunikation.
- Dataopsamling og behandling.
- MQTT til afsendelse af data.
- Opbygning af egen cloud og basal konfiguration til dataopsamling, MQTT-broker og eventuel simpel visualisering.
- Deres egen udvikling under processen, og finde ressourcer til problemløsning. Ræk ud, hvis det er kritisk.

Opsætning af cloud

Studerende har mulighed for at oprette sig hos Webdock med deres studiemail og få op til 3 måneder gratis adgang. Dette kan bruges til at opsætte deres egen cloud-instans til projektet.

Oprettelse kan ske via følgende link:

[Sign up to Webdock](#)

Cloud-miljøet skal som minimum kunne anvendes til at sætte en MQTT-broker op, så enhederne kan sende data til en ekstern platform. Hvis der er tid i projektet, kan grupperne også supplere med en grafisk visning af data, f.eks. et simpelt dashboard eller anden visualisering.

Denne cloud-del giver samtidig mulighed for, at grupperne kan organisere sig mere målrettet. Én eller flere studerende kan have ansvar for cloud og broker, andre kan fokusere på Zephyr-firmware, mens andre arbejder med elektronik, dataopsamling og hardwareintegration. På den måde kan grupperne fordele ansvarsområder, arbejde parallelt og dele læring på tværs af projektet.

Kravs-specifikationer for Zephyr

RTOS og toolchain

- Der skal benyttes Zephyr 4.4.0 da dette er latest stable release.
- Zephyr 4.4.0 og SDK skal matche.
 - [Zephyr-SDK Version Compatibility Matrix - Google Sheets](#)
- Zephyr 4.4.0 understøtter minimum C17, og kode skal derfor være kompatibel med min. C17.

Undtagelse for Projekt C (Nordic Thingy:91 X): For Projekt C opnås Zephyr 4.4.0 gennem nRF Connect SDK (NCS) v3.4.0, som er Nordic Semiconductors officielle fork/overbygning på Zephyr og er baseret på Zephyr 4.4.0 (LTS-release). Dette er et krav, da Thingy:91 X (nRF9151) hverken har board-definition eller cellulær modem-understøttelse i vanilla Zephyr — kun i NCS. Toolchain-matrix-linket ovenfor er derfor ikke relevant for Projekt C, da NCS's Toolchain Manager/nRF Util automatisk installerer den matchende Zephyr SDK (v1.0.1 for NCS v3.4.0).

Udviklingsmiljø

Projektet skal kunne bygges i en af følgende miljøer:

- Host OS (Windows/Linux/macOS).
- VS Code devcontainer ([Developing inside a Container](#))

OBS: !!Ved valg af devcontainer hentes SDK som en del af docker image!!

Docker images kan hentes fra nedenstående link:

[zephyrprojectrtos/zephyr-build - Docker Image](#)

Projekt C: Udviklingsmiljø (Nordic Thingy:91 X)

Det generelle devcontainer-miljø ovenfor (*zephyrprojectrtos/zephyr-build*) er ikke anvendeligt for Projekt C, da det ikke indeholder Nordic HAL, *nrf_modem*-biblioteket, TF-M eller Thingy:91 X's board-filer. De studerende vil derfor ikke kunne bygge til Thingy:91 X i dette miljø. I stedet skal en af følgende to metoder anvendes:

- nRF Connect for VS Code extension + Toolchain Manager (anbefalet, GUI-baseret installation af SDK og toolchain).
- Manuel west-workspace: `west init -m https://github.com/nrfconnect/sdk-nrf --mr v3.4.0` efterfulgt af `west update`. Bemærk at NCS-manifestet trækker Zephyr ind som submodul, i modsætning til Projekt A/B hvor Zephyr er root repo.

Zephyr konfiguration med Kconfig

Projektet skal benytte `prj.conf` til at styre Kconfig, og det giver mulighed for at aktivere subsystems som f.eks.:

- `CONFIG_NETWORKING`
- `CONFIG_NET_SOCKETS`
- `CONFIG_LOG`
- `CONFIG_MQTT`
- Etc.

Kconfig search kan bruges til at få overblik over de indstillinger, som er tilgængelige i den pågældende version. Det kan give et bedre indblik i hvordan subsystems/indstillinger relaterer sig til hinanden internt i Zephyr.

[Kconfig Search — Zephyr Project Documentation](#)

 https://docs.zephyrproject.org/4.4.0/kconfig.html#CONFIG_IMG_ENABLE_IMAGE_CHECK

Når dokumentation gennemses, er det en stor fordel at indsætte versionsnummer, da der kan forekomme forskelle mellem latest/main/4.4.0/4.3.0 osv.

For **Projekt C**: NCS tilføjer sine egne Kconfig-symboler (f.eks. `CONFIG_NRF_MODEM_LIB`, `CONFIG_LTE_LINK_CONTROL`, `CONFIG_MQTT_HELPER`), som ikke findes i den generelle Zephyr Kconfig-søgning ovenfor. Brug i stedet Nordic's egen [Kconfig Search](#) under NCS-dokumentationen, da Projekt C har et større konfigurationsrum end Projekt A/B.

Devicetree

Devicetree (.dts) filer skal anvendes til konfiguration af hardware. Er standard .dts ikke tilstrækkeligt, skal overlay (.overlay) anvendes.

Alle afvigelser fra standard board layout som f.eks. GPIO setup skal dokumenteres.

Devicetree filer kan sandsynligvis benyttes som standard, men hvis der er afvigelser fra den originale devicetree (.dts) fil, så skal det gøres med. overlay filer.

Er der afvigelser i form af custom GPIO-definitioner, så skal det dokumenteres.

For **Projekt C**: Board-devicetree ligger ikke i *zephyr/boards/*, men i nrf-repoet (en del af NCS-manifestet), f.eks. *boards/nordic/thingy91x/*. Vær opmærksom på dette, når der laves overlays, så der ikke ledes forgæves i det forkerte repo.

Zephyrs API'er

Zephyrs interne API'er tillader kommunikation med subsystems, og skal benyttes for projektet. Dette kan gøre sig gældende for bl.a. netværk og modbus.

[API Overview — Zephyr Project Documentation](#)

For **Projekt C**: Cellulær modem-kommunikation på nRF9151 sker ikke via vanilla Zephyrs netværks-/modem-subsystem, men udelukkende via Nordics proprietære [nrf_modem](#)-bibliotek og [LTE Link Controller](#) (*lte_lc*), som kun findes i NCS. Cellulær forbindelse skal etableres gennem disse, da de ikke findes i vanilla Zephyr.

Kernel

Systemet skal benytte Zephyrs kernel funktionalitet:

- Applikationen skal være multithreaded, og kan passende inddeles i 1 thread pr. modul f.eks.:
 - Netværk
 - Applikationslogik.
 - IoT-platform modul.
 - MQTT
- Delte ressourcer hvor flere tråde har adgang til samme data kan med fordel håndteres med semaphore, mutex eller message queue, så den multi-threaded applikation fungerer optimalt.

Arkitektur

I linje med Zephyrs interne struktur skal koden skal være velstruktureret og opdelt i moduler. Det betyder at kode relateret til f.eks. netværkslag har sin egen:

- .c fil (source kode)
- .h fil (Header som eksponerer tilgængelige funktioner)

F.eks.:

- network.c, network.h
- mqtt_client.c, mqtt_client.h
- modbus.c, modbus.h

Mappestruktur

Zephyr projektet skal være struktureret, og eksempel kan se sådan her ud.

```
skylark-nature-app/  
├── app.overlay  
├── CMakeLists.txt  
├── Kconfig.sysbuild  
├── prj.conf  
├── remote  
│   ├── boards  
│   │   └── esp32c6_devkitc_lpcore.overlay  
│   ├── CMakeLists.txt  
│   ├── prj.conf  
│   └── src  
│       └── main_ULP.c  
├── sample.yaml  
└── src  
    ├── main.c  
    ├── modbus.c  
    ├── modbus.h  
    ├── mqtt_client.c  
    ├── mqtt_client.h  
    ├── network.c  
    ├── network.h  
    └── sysbuild.cmake
```


Protokoller

- TLS skal anvendes til sikker kommunikation.
- MQTT skal anvendes til kommunikation med IoT-plattform.
- Modbus RTU/TCP skal anvendes til dataopsamling.
- SparkplugBv1.0 struktur som standard.

Implementeringsdetaljer kan tilpasses for nedenstående:

- Systemet skal benytte sikker kommunikation (TLS).
 - Self-signed eller underskrevne med certbot + Let's Encrypt.

Logging

Zephyrs logging subsystem skal anvendes, og følgende events SKAL logges:

- Netværksstatus
- Cloud forbindelse
- Errors
- Custom logs
- Optional: *Der kan med fordel logges i hver .c fil – supplement til debug.*

Vær obs på log levels.

[Zephyr API Documentation: Logging API](#)

Fælles for dataformat

Alle projekter skal som minimum sende MQTT-beskeder, der indeholder nedenstående. Grupperne kan selvfølgelig supplere med yderligere felter eller målepunkter, hvis det giver mening for det valgte projekt.

- device-id
- timestamp
- Aflæste værdier

Projekt A – Espressif ESP32C6-devkitc

ESP32C6 m. ekstern cellulært modul – cellulær kommunikation

Målet er at enheden kan forbinde sikkert til de studerendes IoT-platform, og at der på enheden kører Zephyr RTOS.

Enheden skal kommunikere cellulært og de studerende skal tænke over lavt strøm- og dataforbrug.

Kravspekificationer

Systemet skal:

- Afvikle Zephyr RTOS jf. generelle krav.
- Etablere cellulær forbindelse til cloud platform gennem udleveret SIM-kort.
- Kommunikere sikkert (TLS).
- Udvikles med fokus på lavt strømforbrug ved hjælp af sleep modes eller interrupts.
- Udvikles med fokus på lavt dataforbrug ved hjælp af batching eller små payloads.
- Systemet SKAL anvende studerendes egen cloud server til dataopsamling.
- De studerende skal som minimum have overvejet gældende standarder som Cyber Resilience Act eller 62443.

Dataopsamling

Opsamling gennem vilkårlig metode

- Samle data med et fast defineret interval (5-15 sek).
- Anvende data i MQTT beskeder.
- SparkplugBv1.0 standard.

Hvad skal data indeholde?

Se [Fælles for dataformat](#)

Projekt B

Industrial IoT ESP32 PLC – Ethernet kommunikation

Målet er at enheden kan forbinde sikkert til de studerendes IoT-platform, og at der på enheden kører Zephyr RTOS.

Lavt strøm- og dataforbrug gør sig ikke gældende, men derimod skal fokus ligge på driftssikkerhed.

Kravspecifikationer

Systemet skal:

- Afvikle Zephyr RTOS jf. generelle krav.
- Kommunikere sikkert over TLS.
- Benytte Ethernet.
- Automatisk reconnect ved tabt netværksforbindelse.
- Systemet skal sende data til IoT-platforme via MQTT.
- Systemet skal kunne håndtere genetablering af drift uden manuel indgriben ved fejl eller reboot.
- Systemet SKAL anvende studerendes egen cloud server til dataopsamling.
- De studerende skal som minimum have overvejet gældende standarder som Cyber Resilience Act eller 62443.

Dataopsamling

Opsamling fra Modbus slave

- Systemet skal kunne opsamle relevant MODBUS RTU/TCP-data fra ekstern enhed.
- Data samles op med et fast defineret interval (5-15 sek.).
- Anvende data i MQTT-beskeder i SparkplugBv1.0 standard.

Hvad skal data indeholde?

Se [Fælles for dataformat](#)

Projekt C

Nordic Thingy:91 X – Cellulær kommunikation

Målet er at enheden kan forbinde sikkert til de studerendes IoT-platform, og at der på enheden kører Zephyr RTOS.

Projekt C udvikles på Nordic Thingy:91 X (nRF9151) gennem nRF Connect SDK ([NCS](#)) v3.4.0, som er baseret på Zephyr 4.4.0 (LTS-release).

Enheden skal kommunikere cellulært og de studerende skal tænke over lavt strøm- og dataforbrug.

Kravspekifikationer

Systemet skal:

- Afvikle Zephyr RTOS jf. generelle krav.
- Etablere cellulær forbindelse til cloud platform gennem udleveret SIM-kort.
- Kommunikere sikkert (TLS).
- Udvikles med fokus på lavt strømforbrug ved hjælp af sleep modes eller interrupts.
- Udvikles med fokus på lavt dataforbrug ved hjælp af batching eller små payloads.
- Systemet SKAL anvende studerendes egen cloud server til dataopsamling.
- De studerende skal som minimum have overvejet gældende standarder som Cyber Resilience Act eller 62443.

NCS bruger [sysbuild](#) som standard build-system (multi-domain: f.eks. både *cpuapp* og evt. *cpunet* for nRF5340-delen på Thingy:91 X til Wi-Fi/BLE). Dette er en anden byggemodel end enkeltimage-build vist i mappestruktur-eksemplet (ESP32C6).

Dataopsamling

Opsamling gennem vilkårlig metode

- Samle data med et fast defineret interval (5-15 sek.).
- Anvende data i MQTT beskeder.
- SparkplugBv1.0 standard.

Hvad skal data indeholde?

Se [Fælles for dataformat](#)

Nice to have

- Enhed kan modtage kommandoer fra cloud gennem MQTT og sparkplugBv1.0.
- OTA firmware opdateringer.
- Grafisk brugerflade til opsamlet data og/eller logs.
- Sikkerhedsfeatures som:
 - mTLS (Hvis man har mod på gensidig autentificering).
 - Bootloader verifikation af firmware.
 - Memory layout diagram, hvis det afviger fra hvad Zephyr devicetree tilbyder for jeres chip.

Getting started

Der stilles screenshots og samples til rådighed, som demonstrerer mappestruktur med logging-eksempel i main.c samt build- og flashkommandoer.

Formålet er at sikre, at alle grupper kan komme i gang med udviklingen efter en industriel standard, som er i linje med Micro Technic's.

Grupper kan derefter selv bygge videre på det fundament, og implementere funktionalitet alt efter valgt/tildelt projekt.

Materialet uploades på itslearning.

Overholdelse af gældende standarder

Fælles for både IEC 62443 og Cyber Resilience Act er at der ikke skal dokumentere en fuld overholdelse, men der skal derimod kunne argumenteres og redegøres for de punkter man har valgt fra enten IEC 62443 eller CRA.

Formålet med dette krav er at kunne arbejde ud fra en industriel tankegang, hvor disse standarder tænkes ind i udviklingsprocessen.

62443

De studerende skal vælge punkter fra standard, som de skal kunne redegøre for, og hvordan de relaterer sig til deres løsning. Det kan f.eks være sikker kommunikation, adgangskontrol, robust fejlhåndtering, logging eller dokumentation af valgte sikkerhedsforanstaltninger.

Standard udleveres af Micro Technic.

Cyber Resilience Act

CRA (Cyber Resilience Act) vil træde i kraft i slut 2027, og fremskriver nogle klart definerede sikkerhedskrav til IoT-produkter. Det vil være relevant for studerende at sætte sig ind i hvad det betyder i udviklingsprocessen, og vælge et par krav som supplement. Grundet kompleksitet er der udvalgt et par stykker her, som skulle være til at gå i kast med (Se [udvalgte paragraffer](#)).

Vi arbejder kun i annex 1 her, fuld tekst kan læses her: [Cyber Resilience Act text, Annex 1 \(15.9.2022\)](#)

Udvalgte paragraffer

- 1.c() *"protect the confidentiality of stored, transmitted or otherwise processed data, personal or other, such as by encrypting relevant data at rest or in transit by state of the art mechanisms;"*
- 1.d() *"protect the integrity of stored, transmitted or otherwise processed data, personal or other, commands, programs and configuration against any manipulation or modification not authorised by the user, as well as report on corruptions;"*
- 1.h() *"be designed, developed and produced to limit attack surfaces, including external interfaces;"*
- 1.j() *"provide security related information by recording and/or monitoring relevant internal activity, including the access to or modification of data, services or functions;"*
- 2.1() *"identify and document vulnerabilities and components contained in the product, including by drawing up a software bill of materials in a commonly used and machine-readable format covering at the very least the top-level dependencies of the product;"*